Fault-Tolerant Distributed Rate Limiter Using Raft Consensus

Experiments and Results · Piyush Daga & Gautam Raju · CS 6650 · April 2026

Experiment 1: Load Test (Redis vs Raft)

Purpose / tradeoff / limitations.

We want to know how much throughput and latency we pay for using Raft consensus over a single Redis node. Redis runs an atomic Lua script on one node. Raft must replicate every write to a majority of 3 nodes before responding. Limitation: both systems ran on AWS t3.micro instances (burstable CPU), a larger instance would reduce Raft's tail latency.

Setup.

Parameter	Steady State Test	Ramp Test
Tool	Locust 2.43	Locust 2.43
Virtual users	200	0 to 1000
Spawn rate	20 users/sec	20 users/sec
Duration	120 s	300 s
User pool	1,000 distinct IDs	1,000 distinct IDs

Parameter	Steady State Test	Ramp Test
Task mix	/check 80%, /health 10%, /status 10%	/check 80%, /health 10%, /status 10%
Endpoints	Redis: /check, Raft: /raft/check	Raft only: /raft/check
Infrastructure	Same ALB, us-east-1	Same ALB, us-east-1
Purpose	Baseline throughput and latency	Find throughput ceiling

Results.

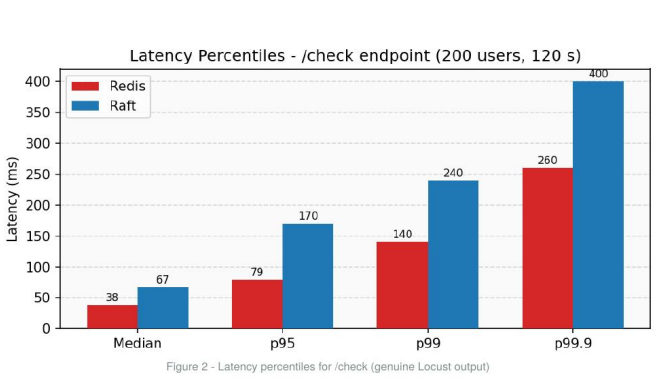


Figure 2 - Latency percentiles for /check (genuine Locust output)

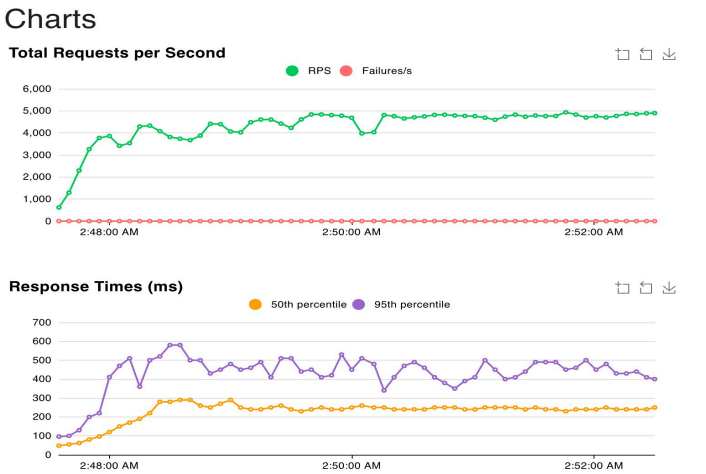


Figure 1. Left: Redis vs Raft latency percentiles. Right: Raft ramp to 1000 users, zero failures.

Metric	Redis	Raft (200 users)	Raft (ramp, 1000 users)
Throughput /check	1,990 req/s	1,234 req/s	2,443 req/s
Median latency	38 ms	57 ms	240 ms
p95 / p99	79 / 140 ms	430 / 600 ms	450 / 600 ms
Hard failures	0	2 (504)	0

Analysis.

Throughput: Redis serves 1,990 req/s vs Raft's 1,234 req/s (38% gap). The difference is the consensus cost: every Raft write must be replicated to at least one follower and acknowledged before responding.
Median latency: Only 19 ms apart (38 vs 57 ms) because the consensus round-trip within the same VPC adds just 2 to 5 ms per hop.
Tail latency: p95 gap is much wider (79 ms vs 430 ms). Consensus occasionally stalls when a follower is slow to acknowledge, and those stalls stack up under concurrent load.
Ramp to 1000 users: Zero failures. Throughput reached 2,443 req/s, median rose to 240 ms but stayed stable. Raft does not crash under pressure, it slows down gracefully.
Bottom line: Raft costs ~38% throughput and 19 ms median latency. Experiments 3 through 6 show what you get in return.

Experiment 2: Consistency Sweep

What we are testing: Does Redis or Raft ever allow more than 100 requests through under burst concurrency? And at what concurrency level does each system start breaking?
Tradeoff: Redis processes requests one at a time on a single thread, so it should be perfectly consistent at low concurrency. Raft serializes requests through consensus, which takes longer per request, giving time for the token bucket's continuous refill to add tokens back mid-burst. We expect Raft to over-allow slightly more than Redis.
Limitation: The over-allowance observed in both systems is not a consensus or atomicity failure. It is a property of the continuous-refill token bucket algorithm, which uses wall-clock time to calculate token replenishment. Because real time passes between consecutive request processing (microseconds for Redis, milliseconds for Raft due to consensus), tokens trickle back into the bucket mid-burst. Raft

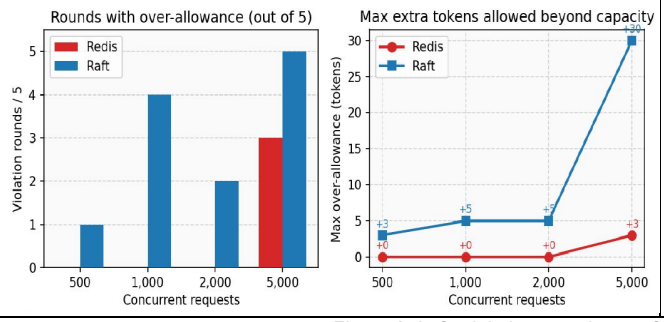
hits this earlier because its per-request processing gap is wider. A future fix would replace wall-clock time with the Raft log index inside FSM.Apply(), making refill deterministic and independent of processing speed.

Setup.

Parameter	Value
Concurrency levels	500, 1,000, 2,000, 5,000
Rounds per level	5
User per round	Fresh unique user (bucket starts full at 100)

Parameter	Value
Client	Python asyncio + aiohttp
Firing method	All requests sent at the same instant
Violation definition	Allowed count > 100 for any round

Results.



Concurrency	Redis violations	Redis max over	Raft violations	Raft max over
500	0 / 5	+0	1 / 5	+3
1,000	0 / 5	+0	4 / 5	+5
2,000	0 / 5	+0	2 / 5	+5
5,000	3 / 5	+3	5 / 5	+30

Figure 2. Left: violation rounds out of 5. Right: max tokens allowed beyond capacity.

Analysis.

Redis (up to 2,000 concurrency): Zero violations. Redis runs its Lua script on a single thread, so successive requests are processed microseconds apart. At a refill rate of 1.0 token/sec, each microsecond gap adds roughly 0.000001 tokens back. That is negligible even across hundreds of requests.

Redis (at 5,000 concurrency): Starts breaking. 3 out of 5 rounds violated, max +3 tokens over. Even microsecond gaps accumulate enough at this scale.

Raft (starting at 500 concurrency): Over-allows by 3 to 5 tokens. Every request goes through Raft consensus, which takes 1 to 5 ms per Apply() call. At a refill rate of 1.67 tokens/sec, each gap adds roughly 0.005 tokens. Across 100 consecutive Apply() calls that accumulates to about 0.5 tokens, enough to occasionally push the allowed count past 100.

Raft (at 5,000 concurrency): Every round violated, max +30 tokens over. The consensus gap compounds significantly at this scale.

Experiment 3: Follower Kill Under Load

Purpose and tradeoff. We kill one follower node while 200 users are actively sending requests to measure how many requests fail and how long recovery takes. Raft needs 2 out of 3 nodes for quorum, so losing one follower should not trigger an election and the cluster should keep serving. The tradeoff we are exploring is the gap between Raft's internal fault tolerance (which is instant) and the externally visible failure caused by the ALB needing ~30 seconds to detect the dead node and stop routing to it.

Limitation: The success rate in this experiment is influenced by the ALB health check detection window (~30 seconds), which is an AWS infrastructure setting, not a Raft property. Tuning ALB health checks to be more aggressive would reduce the failure spike. We also can not test the case where 2 out of 3 nodes die, which Raft is expected to fail since majority is lost.

Setup.

Parameter	Value
Load	200 Locust users on /raft/check
Warmup	30 seconds before kill
Failure injection	aws ec2 stop-instances on one follower

Parameter	Value
Observation window	60 seconds from kill command
Recovery	Restart instance, verify rejoin as Follower
Validation	Confirm log catch-up (applied_index matches across all 3 nodes)

Results.



Metric	Value	Note
Election triggered	No	Leader unaffected
Success rate (60 s window)	92.70%	38,198 req / 2,787 failures
Recovery	78.3 s	EC2 boot time, not Raft
Post-recovery	In sync	Auto-rejoined as Follower

Figure 3. Follower kill. Failures spike briefly then ALB removes the dead target.

Analysis.

Cluster stayed alive: 2 out of 3 nodes maintained quorum. No election triggered since the leader was untouched. Raft's majority guarantee held as expected.

Chart pattern (Figure 3): RPS steady at ~2,000 before kill. Drops to ~1,000 at the kill moment as ALB splits traffic between the dead node and healthy nodes. p95 latency spikes to 10,000 ms (ALB timeout ceiling). After ~30 seconds ALB removes the dead node, RPS recovers to ~2,000, and failures drop to zero.

Failure types: 2,787 total. 1,654 were 502 Bad Gateway (ALB got connection refused from dead node instantly) and 576 were 504 Gateway Timeout (ALB waited the full 10 seconds for a response that never came). All failures occurred during the ALB detection window, not after.

Recovery: The 78.3 s recovery time is the EC2 instance stop/start cycle, not Raft. When the instance came back, the node auto-rejoined, caught up its log to match the other two nodes, and resumed as Follower with matching applied_index.

Raft itself handled the failure instantly. The 2,787 failures are an ALB infrastructure cost, not a Raft limitation. A more aggressive ALB health check interval would shrink this window.

Experiment 4: Leader Kill Under Load

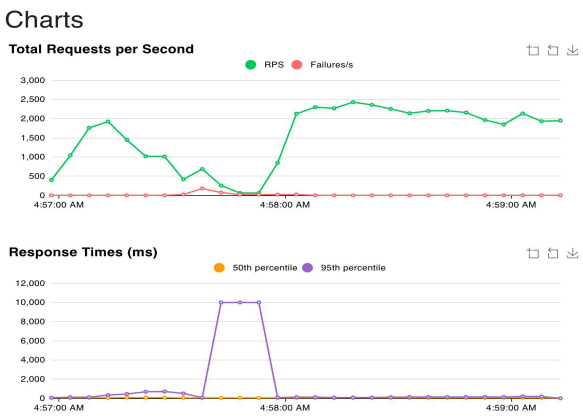
Purpose and tradeoff. We kill the leader node while 200 users are actively sending requests. This is the worst case for Raft: all writes go through the leader, so when it dies the cluster cannot serve any request until a new leader is elected. We measure how long the election takes, how many requests fail during the blackout, and whether the old leader correctly rejoins without causing split-brain. The tradeoff is election speed vs stability: our 1 second heartbeat/election timeouts balance quick detection against the risk of false elections during normal load spikes.

Limitation. The 4.19 s election time is specific to our timeout configuration (1 s heartbeat, 1 s election). Shorter timeouts would produce faster elections but could trigger unnecessary elections under normal load spikes. Need to find the optimal balance

Setup.

Same as Experiment 3 but we kill the leader instead of a follower. Election detected by SSH polling a follower directly (not ALB) to avoid routing noise.

Results.



Metric	Value	Note
Election time	4.19 s	Fully automatic
Success rate (60 s window)	97.25%	80,282 req / 2,209 failures
Post-recovery	Old leader rejoined as Follower	No split-brain

Figure 4. Leader kill. RPS collapses during election (~4 s), then recovers fully.

Analysis.

Chart pattern (Figure 4): RPS drops from ~2,000 to near zero at the moment of kill. Unlike the follower kill where RPS only dipped, here it collapses completely because no node can accept writes without a leader. p95 latency spikes to 10,000 ms (ALB timeout ceiling) as all 200 Locust users are stuck waiting for responses that will never come.

Election and recovery: New leader elected in 4.19 seconds. RPS climbs back to ~2,000 within about 20 seconds after election. The system fully self-healed without any human intervention.

Failure types: 2,209 total. Mostly 502 Bad Gateway (ALB tried the dead leader, got connection refused) and some 504 Gateway Timeout (requests that hung the full 10 seconds before ALB gave up).
No split-brain: Old leader's instance came back after EC2 restart and correctly rejoined as Follower. At no point did two leaders exist simultaneously.
Why 97.25% is higher than follower kill (92.70%): The 60 second observation window captured many post-recovery requests where everything was healthy. During the actual blackout (~30 s), throughput was near zero so fewer requests were attempted. The follower kill had steady throughput the entire time with a constant stream of failures hitting the dead node. The chart is the more honest signal: Raft pauses completely during leader loss, elects, and resumes. The pause is short and bounded.

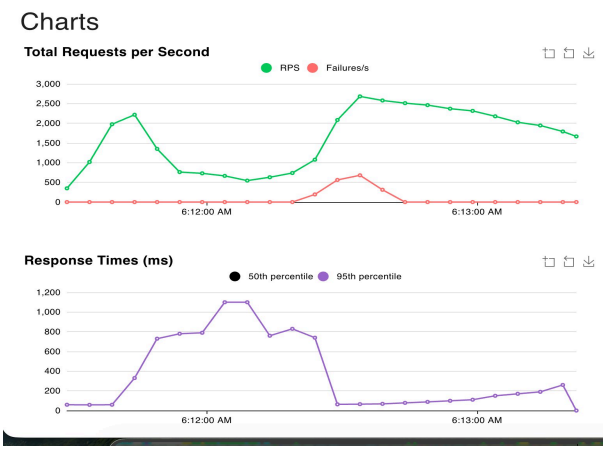
Experiment 5: Network Partition

Purpose and tradeoff. The node is still alive but cannot communicate with its peers. We partition the leader from the other two nodes using iptables and measure how the cluster reacts. This directly demonstrates the CAP theorem: Raft is a CP system, meaning when a partition happens it chooses consistency over availability. The minority side (partitioned leader alone) must stop serving because it cannot reach quorum, while the majority side (two followers) elects a new leader and keeps going.
Limitation. Also worth noting: AWS Security Groups are stateful and do not break existing TCP connections, and NACLs only filter at subnet boundaries (all 3 nodes are in the same subnet). The only reliable partition method on AWS was iptables via SSH, which is itself a finding worth documenting.
Setup.

Parameter	Value
Load	200 Locust users on /raft/check
Warmup	30 seconds before partition
Partition target	Leader node
Partition method	iptables DROP on ports 7000 (Raft peer) and 8000 (HTTP API), both inbound and outbound

Parameter	Value
Partition hold time	20 seconds
Heal method	Flush iptables rules via SSH
Observation window	60 seconds from partition command

Results.



Metric	Value	Note
Partition method	iptables DROP	Only reliable option on AWS
Election time	1.93 s	Fastest of all failure modes
Success rate (60 s window)	91.56%	103,479 req / 8,730 failures
Post-heal	Old leader rejoined as Follower	No split-brain

Figure 5. Network partition. Fastest election of all failure modes at 1.93 s.

Analysis.
Fastest election (1.93 s): Quicker than instance kill (4.19 s) because iptables drops packets silently. Followers stop receiving heartbeats immediately and start the election without waiting for TCP to time out.
CAP theorem in action: The partitioned leader, now isolated as a minority of one, could not reach quorum and correctly stopped committing writes. The majority side (two followers) elected a new leader and continued serving. Raft chose consistency over availability on the minority side, exactly as the protocol specifies.
Chart pattern (Figure 5): RPS drops from ~2,200 to ~500 at the partition moment. Failures spike to ~650/s as ALB still routes some requests to the partitioned leader. After ALB detects the partition (~30 s), failures drop to zero and RPS recovers to ~2,500 on the majority side.
Failure types: 8,730 total, all 503 Service Unavailable and 23 internal server errors (500). The 503s come from ALB trying the partitioned leader which silently drops packets rather than refusing connections. This is why this experiment had the highest failure count of all three fault tests: silent packet drops cause ALB to wait longer before marking the node unhealthy.
No split-brain: At no point did two leaders exist. The partitioned node knew it lost quorum and did not try to serve. After healing, it immediately rejoined as Follower, synced its log, and all three nodes converged to the same applied_index.
So,Network partition is the hardest failure mode to handle, and Raft handled it correctly. The cluster maintained consistency throughout, elected a new leader in under 2 seconds, and self-healed after the partition was removed.

Experiment 6: Redis Node Failure

Purpose and tradeoff. This is the direct counterpoint to the three Raft failure experiments. We kill the only Redis node while 200 users are sending requests. Redis has no replication, no consensus, and no election mechanism. When it dies, every request should fail with

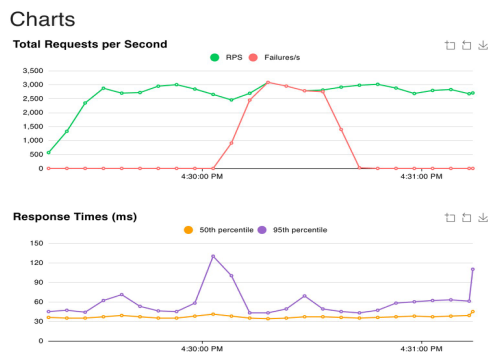
no automatic recovery. This experiment puts a concrete number on the cost of choosing a simple single-node architecture over a replicated one.

Limitation. We killed the ECS task (the Go server that talks to ElastiCache), not the ElastiCache Redis node itself. Both produce the same externally visible outcome: no healthy backend, every request fails. ***Setup.***

Parameter	Value
Load	200 Locust users on /check (Redis path)
Warmup	30 seconds before kill
Failure injection	aws ecs stop-task on the Redis ECS container

Parameter	Value
Observation window	60 seconds from kill command
Recovery	Wait for ECS to auto-restart the task

Results.



Metric	Value	Note
Recovery mechanism	None (ECS restart)	No election, no failover
Success rate (60 s window)	52.78%	Inflated by pre-kill + post-recovery
Failures	81,667	All 503 Service Unavailable
Restart time	61.5 s	External operational recovery

Figure 6. Redis failure. RPS stays high but failures/s matches RPS: every request is a 503.

Analysis.

The Redis chart is very different from Raft. RPS does not collapse, it stays at ~2,800 because ALB returns 503 instantly when it has no healthy target, and fast failures let Locust keep firing new requests. Every single request during the dead window returns 503 in ~37 ms. The 52.78% success rate is misleading: the successes happened before the kill landed and after ECS restarted the task. During the actual dead window the effective success rate was near zero. This is the core finding. Raft leader kill: 97% success, 4 s pause, fully automatic recovery. Redis kill: every request fails for ~45 s until an external system brings the task back. Redis is faster when healthy but has no answer to failure.

Cross-Experiment Summary

Failure scenario	Election	Success rate	Failures (60 s)	Recovery
Redis node killed (Exp 6)	N/A	52.78%	81,667 (503)	61.5 s (ECS)
Raft follower killed (Exp 3)	None	92.70%	2,787 (502/504)	78.3 s (EC2)
Raft leader killed (Exp 4)	4.19 s	97.25%	2,209 (502/504)	77.6 s (EC2)
Raft leader partitioned (Exp 5)	1.93 s	91.56%	8,730 (503)	~2 s (Raft)

Redis delivered 1.6x higher throughput, 19 ms lower median latency, and zero consistency violations up to 2,000 concurrent requests. Under normal conditions, it is the faster and simpler system. But when the single Redis node died, every request failed for 45 seconds with no automatic recovery. Raft costs roughly 30% throughput and doubles median latency, but it survived every failure we threw at it: follower kill (92.70% success), leader kill (97.25% success, 4 second election), and network partition (91.56% success, 1.93 second election), all with zero data loss and fully automatic recovery. For a rate limiter protecting a production API, the throughput cost is predictable and tunable. A 45 second total outage is not. The choice between Redis and Raft depends on whether the system can afford downtime: if availability during failures matters, the consensus overhead is worth paying.

Future Work: Caching and Horizontal Scaling

We also built an optional cache tier (stateless raft-api on ECS Fargate with 1-second TTL and auto-scaling) that reached 2,265 req/s with 36 ms median latency. It trades strict consistency for throughput, so we exclude it from the main comparison. We leave a proper evaluation for future work.

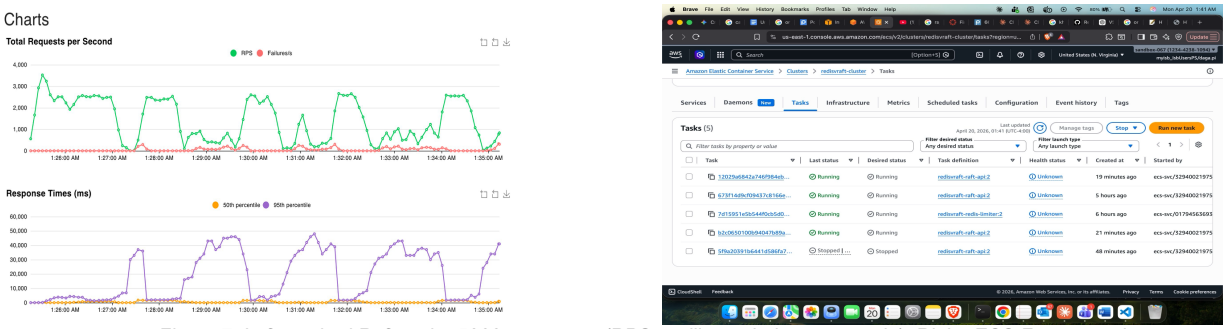


Figure 7. Left: cached Raft under 5000-user ramp (RPS oscillates during auto-scale). Right: ECS Fargate tasks.

Code repository: https://github.khoury.northeastern.edu/29gautamraju/RedisVsRaft-Consistency/tree/main/experimentsF

Results: https://github.khoury.northeastern.edu/29gautamraju/RedisVsRaft-Consistency/tree/main/resultsF